Thetawave.ai

💻 軟體架構設計 – 門面模式與封裝變化原理

Generated by Thetawave.ai

6.1 基礎架構思維、模式及邊界劃分.mp4

🏗️ 什麼是結構型模式（Structural Pattern）

> **結構**指的是「互相關聯的元素排列」，其中關鍵是「元素」與「元素之間的關係」。
> 抽象化來看，結構就像由「點與線」組成的圖形：
>
> **點**代表元素（在物件導向中即為物件）
>
> **線**代表元素之間的關係

物件導向視角下的結構

層級	範例與說明
高階結構	宇宙、天空、地球
中階結構	地球上的國家、鄉村、城市、道路
低階結構	城市內的建築、房屋、室內格局（客廳、臥室、廚房等）
最低結構	房間內的物件（書桌、床、衣櫃、窗戶的玻璃、框架等）

在物件導向中，**結構 = 物件（點）+ 物件之間的關係（線）**，也可視為「類別及類別之間的關係」。

繪製類別圖（Class Diagram）就是為了捕捉這種物件導向的結構。

🔗 結構型模式關注的核心：減少耦合（Decoupling）

- 結構型模式不像行為型模式著重物件「行為」，而是著重類別圖的「格局」—即類別與類別間的耦合度（Coupling）。
- 透過結構型模式，我們能有效**降低系統的耦合度**，讓系統更穩固且易於維護。

🏛️ 門面模式（Facade Pattern）

一、背景與需求

你接到一個套件開發案，要提供一個能解析 Markdown 語法表格的套件，並且統計表格中的資料（例如：計算總花費 Total Cost，計算成員數量 Count Members），同時要允許後續擴充各種不同的統計操作。

二、系統設計與問題

類別名稱	職責說明
Markdown Parser	解析 Markdown 格式的表格，輸出 Table 物件
Table	儲存表格資料（Header 與 Data），提供存取方法
Table Stats Performer	接受統計策略，對 Table 進行統計並印出結果
Stats Operation 介面	定義統計操作的行為（如加總、計數等），以策略模式實作
具體統計策略類別	例如 Total Column（加總某欄位）、Count Rows（計數列數）

- **策略模式（Strategy Pattern）**用於封裝統計行為，使擴充統計操作變得容易且符合開放封閉原則（OCP）。
- 但此設計導致**Client 端必須認識多個介面（Interface）和類別**，使用門檻高且複雜。

三、介面（Interface）定義與理解

> 「介面」是類別與使用者（Client）之間的「那一面」，是一種抽象的契約，定義了類別對外公開的操作，但不包含內部實作細節。

- 介面是 Client 與類別間的溝通橋樑
- Client 只能依賴介面，無法直接操作類別內部細節
- 介面的數量與複雜度影響 Client 的學習曲線與使用便利性

Thetawave.ai

⚔️ 兩道衝突的 Forces（設計壓力）

Forces	說明
高結構複雜度（High structural complexity）	系統內有多個介面相互依賴，結構複雜且不規則，增加理解與使用難度
良好易用性（Good usability）	Client 希望有簡單易用的介面，降低學習門檻與使用複雜度

🛡️ 解決方案：設計門面（Facade）

1. 門面（Facade）定義

> 「門面」字源於法文 facade，指建築物的正面。
> 門面模式即是為複雜系統設計一個簡潔、統一的介面，讓 Client 能輕鬆使用，而不必了解系統內部的複雜結構。

2. 門面模式的做法

步驟	說明
建立門面類別	建立一個名為 `StatFacade` 的類別，封裝系統內部的複雜結構
提供簡潔操作	在 `StatFacade` 上定義明確的操作（如 `premark_on_table_stats`）供 Client 使用
指導 Client 使用	Client 僅依賴 `StatFacade` 類別，無需直接接觸內部多個介面

3. Client 使用方式示例

Thetawave.ai

```
StatFacade facade = new StatFacade();
facade.premark_on_table_stats("data.table", TotalCost | CountMembers);
```

- 使用位元 OR 運算子（bitwise OR）組合多個統計選項（TotalCost 和 CountMembers），提高可讀性且降低耦合。

🔍 門面模式的優缺點

優點	缺點
大幅降低 Client 對系統複雜度的感知，提高生產力與維護性	門面類別必須配合系統結構變動而維護，維護成本與內部變動頻率成正比

總結：門面模式的核心價值

門面模式透過封裝系統複雜結構並提供簡潔介面，**成功解耦 Client 與系統內部結構的關係**，讓使用者能在完全不知系統細節的情況下輕鬆達成目標。這不僅提升了系統的易用性，也使得軟體設計更符合模組化與維護性的要求。

門面模式在第三方套件與框架開發中尤其重要，因為易用性往往決定了套件的普及度與實際應用價值。

介面（Interface）定義與理解回顧

詞彙	說明	英文詞根與意義
Interview	面試、面談	The view between us（我們之間的面）
Interchange	交換	The change between us（我們之間的改變）

Interact	互動	The action between us（我們之間的行動）
Internet	網路	The network between us（我們之間的網路）
Interface	界面、介面	The face between us（我們之間的那一面）

介面是 Client 與類別之間的「臉面」，是雙方溝通的橋樑，隱藏了內部複雜性。

策略模式（Strategy Pattern）在本案例的應用

角色	職責說明
Stats Operation 介面	定義統計操作的行為（如 perform 方法）
Total Column	實作加總某欄位的統計行為
Count Rows	實作計算列數的統計行為
Table Stats Performer	將具體統計策略注入，對 Table 執行統計並輸出結果

策略模式成功解決了「行為變動性」及「擴充性」兩大需求，讓統計操作能靈活擴充，且符合開放封閉原則。

Client 與系統介面的複雜關係示意

角度	介面數量	影響
系統設計者	1（Stats Operation）	定義統一的統計操作介面
Client 使用者	7 個介面	需理解與操作多介面，學習門檻高，使用複雜度大

物件導向分析與設計（OOAD）於本案例的角色

- 每個類別專注單一職責，降低複雜度
- 透過策略模式封裝變動行為
- 使用門面模式降低系統對 Client 的暴露，簡化使用流程
- 運用介面定義契約，促進模組間解耦

Thetawave.ai

重要關鍵詞總結

關鍵詞	定義與說明
結構（Structure）	元素（物件）與元素間關係（連結）的排列組合
耦合度（Coupling）	類別間依賴關係的強弱
解耦（Decoupling）	降低系統內部元素間的依賴，提高系統彈性與維護性
介面（Interface）	類別對外暴露的操作集合，定義了使用者能調用的行為
策略模式（Strategy）	封裝行為變動，讓行為可靈活替換且符合開放封閉原則
門面模式（Facade）	為複雜系統提供簡潔介面，隱藏內部結構，降低使用複雜度
結構複雜度（Structural Complexity）	系統中介面與類別數量多且彼此依賴複雜，增加理解與使用難度
易用性（Usability）	系統對使用者友善程度，影響學習曲線與使用效率

實務啟示

門面模式強調從使用者角度設計介面，將系統內部複雜度封裝後暴露簡化介面，使得系統既保持良好的模組性，也能降低學習與使用門檻。這種設計理念在軟體工程中極具價值，尤其適用於開發大型、複雜度高且需廣泛被使用的套件或框架。

多數情況下，系統內部結構變動會引起門面類別的修改，因此門面模式的維護成本與系統結構變動頻率密切相關。如何平衡維護成本與易用性，成為設計者必須思考的重點。

6.3 基礎架構思維、模式及邊界劃分.mp4

💡 軟體架構學的核心觀念

> **軟體架構**是關於如何設計軟體系統的整體結構，使其能夠隨著市場需求的變化，方便且低成本地修改與擴充。

軟體本質與架構設計目標

重點	說明
軟體本質	軟體是「軟」的，意即要容易修改（好改）、維護（好維護）、擴充（好擴充）。
高價值軟體	必須能夠高頻率地因應市場需求變化，不斷升級與新增功能。
軟體架構設計目標	每次需求變更時，**修改最少量的類別（Class）與模組（Module）**，降低維護與開發成本。

軟體架構設計的兩大方針

方針	說明
封裝變化	將可能一起變動的部分封裝在同一處，避免改動散布各處。
單向穩定依賴	依賴關係應該從易變的模組指向穩定的模組，維持依賴單向且朝向穩定方向。

封裝變化的顆粒度

顆粒度	說明
類別（Class）	最小的封裝單位，代表系統中的一個實體或概念。
模組（Module）	將一組類別集合起來，形成一個獨立元件，封裝較大範圍的變化。

Thetawave.ai

以「道爾夫撲克牌遊戲」為例理解需求變化

需求變化脈絡分析

遊戲中主要涉及以下幾個核心實體與其責任：

實體	主要責任	變化可能的方向及原因
玩家	遊戲互動與流程，玩家定義與屬性	改變玩家數量、屬性、擴充互動方式
遊戲	管理整體遊戲流程	流程規則變動
牌	手牌的基本單位，由花色和數字定義	增加特殊牌（如鬼牌）、擴充牌的屬性
牌堆	洗牌與發牌，支援遊戲的隨機性	洗牌規則變更、隨機性優化
牌型	出牌規則與組合策略，如單張、對子、順子等	擴充更多牌型、變更出牌規則

需求變化關鍵原因

需求的變化主要圍繞幾個**關鍵原因（關鍵設計思想）**：

變化原因	代表系統中的責任/目標	影響的範圍與模組
隨機性	透過牌與牌堆實作遊戲的隨機性	牌與牌堆類別及相關模組
互動性與流程	玩家之間互動與遊戲流程控制	玩家與遊戲類別
策略性豐富度	透過多樣化的牌型與出牌規則提高策略性	牌型及其出牌規則的類別與模組

Thetawave.ai

物件導向分析（OOA）與封裝變化

- **OOA的SOP第一步**：辨識需求文件中的具有特定意義的詞彙（專有名詞）。
- 這些專有名詞代表系統中的主要實體與複雜度來源，通常也是未來需求變化的焦點。
- 透過OOA萃取出的**類別、責任與關鍵原因**，自然完成了封裝變化的第一層級（類別層級）。
- 因此，良好的OOA分析本質上就是在封裝主要變化範圍，有助於後續架構設計。

模組化設計的原則與策略

模組化設計目標

- 讓需求變更時，修改影響範圍縮小在最少數的模組中。
- 將「一起變的、變法相同的類別」放在同一模組中。

模組化設計步驟

1. **分析需求變化的關鍵原因**，歸納類似變化頻率與方式的類別。
2. **分類模組**，例如核心模組與擴充模組。
3. 在擴充模組中，依使用者重用需求將模組進一步拆分成多個插件模組。

常見的模組分類

模組類型	內容說明	範例
核心模組	定義系統的關鍵概念與核心競爭力，變化較穩定	撲克牌遊戲中的「牌」、「牌堆」、「玩家」、「遊戲」等核心類別

擴充模組	提供豐富的選項與擴充性，變化較頻繁	各種牌型的出牌規則實作（如同花順、三條等）插件模組
應用端模組	負責使用者介面（UI）及使用者體驗（UX）	撲克牌遊戲的前端APP模組

插件模組的劃分示例

插件模組類型	說明	例子
常見排行模組	內建常用的牌型排行	單張、對子、葫蘆、順子、鐵枝
非常見排行插件	獨立模組，便於用戶按需引入	同花、三條、一條龍等特殊牌型

設計模式與模組依賴方向的選擇

依賴方向的兩種設計方案

設計方案	抽象類別/介面位置	依賴方向	缺點
設計一	抽象放在衍生模組	核心模組依賴衍生模組	核心模組不穩定，變動大，影響整體系統穩定性
設計二（推薦）	抽象放在核心模組	衍生模組依賴核心模組	依賴方向符合穩定性原則，核心模組穩定，衍生模組可自由擴充

單向穩定依賴原則（Single Direction Stable Dependency）

> 「依賴關係應該永遠朝向較穩定（變化較少）的模組，避免核心模組依賴易變模組。」

- **核心模組**：定義系統的穩定知識與關鍵概念，變化頻率低。
- **衍生模組**：較為輔助且易變，依賴核心模組以維持整體穩定。

- 抽象類別或介面應放在核心模組，使衍生模組實作並依賴該抽象。

etawave.ai

分層架構的自然浮現

以大老二撲克牌遊戲為例：

層級	內容說明	依賴關係
核心模組	牌、牌堆、玩家、遊戲流程等核心領域模型	穩定，提供抽象介面
插件模組	各種牌型行為的實作，依賴核心模組的抽象	依賴核心模組
應用端模組	使用者介面（UI/UX）、應用流程，依賴核心與插件模組	依賴核心與插件模組

- 變化主要發生在插件模組（擴充牌型）與應用端模組（UI/UX變更）。
- 核心模組保持穩定，避免頻繁變動影響整體系統。

封裝變化與依賴原則的整合意義

- 透過**封裝變化**，將需求變化集中管理，降低修改範圍。
- 透過**單向穩定依賴**，確保依賴關係朝向穩定模組，提升系統穩定性與可維護性。
- 兩者結合帶來的結構是**清晰分層**且**模組化良好**的軟體架構，能有效因應需求頻繁變動的挑戰。

軟體架構底層邏輯與模組分層 🧬

> **軟體架構的最大目的**是將需求變化的影響範圍降至最低。

變化封裝與模組化設計

- **封裝變化**：將需求中會變動的部分集中管理，避免變化波及整個系統。

- 使用 **物件導向分析（Object-Oriented Analysis, OOA）** 把相同責任和改變原因的類別聚合在同一模組中。
- 形成兩個層級的變化封裝：
  1. **類別層級**：將職責收攏於類別。
  2. **模組層級**：將類別依變化原因歸類於模組。

單向穩定依賴原則（Stable Dependency Principle）

- 依賴關係必須遵守「**由易變模組指向穩定模組**」的原則。
- 易變的模組放在外層，穩定的核心模組放在內層，降低核心模組被頻繁變動影響的風險。
- 此原則確保核心模組的**長期穩定性與高維護性**。

四層模組分層架構

層級	功能說明	依賴方向
大老二模組	系統核心與排行相關的模組，通常包含業務邏輯和核心功能	穩定模組
插件模組	提供額外功能的擴充模組	依賴大老二模組
門面模組（Facade）	統整核心和插件模組，提供簡化且統一的介面給前端或外部系統使用	依賴大老二與插件模組
APP模組	最外層的應用程式模組，易變且直接使用門面模組的接口	依賴門面模組

- 門面模組負責隱藏複雜細節，提供簡單易用的接口，讓前端APP能快速整合功能。
- 依賴關係是**單向且穩定的**，由外向內依賴。

應用場景示例：前後端分層

層級	技術角色	功能描述
大老二模組	後端	實作業務邏輯與排行核心功能，部署於後端

門面模組	後端API服務	提供統一API介面，讓前端透過API存取核心功能
APP模組	前端	使用API介面透過門面模組整合功能

- 前端開發者專注於利用簡化接口，不需關心後端內部複雜實現。
- 後端架構透過模組分層與門面模式，有效隔離易變需求。

軟體架構設計的核心兩句話

> **「揮一起變的，放一塊，永遠往穩定的方向依賴。」**

- **揮一起變的**：將會同時改動的部分聚合在同一模組，減少改動範圍。
- **放一塊**：封裝變化，避免變化散落在系統多處。
- **永遠往穩定的方向依賴**：確保依賴關係從不穩定（易變）模組指向穩定模組，保護核心。

門面模式（Facade）在架構中的角色

- 門面模式是一種**設計模式**，用於對外提供簡單且統一的接口。
- 在軟體架構中，門面模組：
  - **整合多個核心及插件模組的功能**。
  - 提供給前端APP一個**易用的接入點**，隱藏內部複雜度。
  - 是易變APP模組與穩定核心模組之間的橋樑。

重要概念與關鍵詞整理

概念	解釋

模組（Module）	將相關類別或功能封裝在一起，便於維護與管理
OOA（物件導向分析）	分析需求與系統中物件的責任，將相同責任的類別聚合
單向穩定依賴原則	易變模組依賴穩定模組，避免核心模組被頻繁改動影響
門面模式（Facade）	提供簡化接口，統一對外服務，隱藏複雜的內部細節
核心模組	系統中最穩定、維護成本最低的模組，承載業務邏輯與關鍵功能
易變模組	需求變動頻繁的模組，置於架構外圍，減少對核心模組的影響

MetaWave.ai

變化封裝的比喻說明 ✨

比喻： 把需求變化想像成「火山爆發」，封裝變化就像是用防火牆將火山爆發的岩漿限制在特定區域，避免燒毀整個城市。模組化設計和單向穩定依賴原則就是建造這道防火牆的策略，讓核心系統保持安全穩定。

軟體架構設計的實踐建議

- 在設計時，先識別需求中哪些部分最易變，哪些最穩定。
- 透過 OOA 將變化相關的職責聚合，形成模組。
- 依照單向穩定依賴原則，規劃模組間依賴方向。
- 利用門面模組封裝複雜性，對外提供簡化接口。
- 持續以需求變化為導向調整架構，避免過度拘泥於嚴格規範。

軟體架構的本質在於**控制變化的影響範圍**，使系統能夠適應不斷變動的需求，同時保持核心功能的穩定與可維護性。